

Veículo: Carro

Grupo: Beatriz Amorim Fragozo, Beatriz Costa Chiara, Agatha Ester Rodrigues Ribeiro, Heloisa Manuela Xaves Romero e Glória Maria Lopes da Silva.

Para a preparação inicial do nó rpi-106, utilizou-se um adaptador de cartão SD em formato de pendrive conectado ao computador da bancada para realizar a instalação do sistema operacional no cartão micro SD através do software Raspberry Pi Imager. No programa, foi selecionado o modelo de dispositivo Raspberry Pi 3B+ e a distribuição Raspberry Pi OS Lite (64-bit) - Legacy, opção recomendada devido à ausência de interface gráfica e por priorizar o desempenho em tarefas de servidor. Em seguida, com o cartão SD selecionado, foram configuradas as opções avançadas de personalização. Definiram-se o nome de host (hostname) como rpi-106 e a ativação da autenticação por SSH para acesso remoto. Foram configurados também o usuário pi (senha 147), o fuso horário de São Paulo e o mapa de teclado no padrão brasileiro. Após a validação dos parâmetros, iniciou-se a gravação e verificação do sistema no cartão. Concluída a gravação, procedeu-se à montagem do hardware: o cartão micro SD foi inserido no Raspberry Pi rpi-106, conectaram-se os cabos de vídeo HDMI e teclado (retirados do computador da bancada) e a fonte de alimentação foi plugada à tomada. Após a inicialização, o sistema exibiu a interface em modo texto no monitor. No computador principal, validou-se a conectividade executando o comando `ping rpi-106` no terminal, seguido pelo acesso remoto via `ssh pi@rpi-106`. Com os testes finalizados, o nó rpi-106 foi desligado e os periféricos retornaram ao computador da bancada. Para a execução dos procedimentos avançados de infraestrutura, conteinização e mensageria previstos nas etapas seguintes da disciplina, utilizou-se o nó rpi-104. Acessou-se o dispositivo via SSH através do comando `ssh pi@rpi-104` (senha 147). A primeira intervenção no sistema consistiu no dimensionamento do armazenamento por meio do utilitário de configuração `sudo raspi-config`, navegando até 6 Advanced Options -> A1 Expand Filesystem. A expansão do sistema de arquivos é fundamental para garantir o aproveitamento total da capacidade do cartão SD, evitando escassez de espaço no carregamento de imagens Docker. O sistema foi reiniciado via `sudo reboot` e, após a reconexão SSH, executaram-se os comandos `sudo apt update` e `sudo apt upgrade -y` para atualizar as listas de pacotes e o sistema operacional. Na sequência, realizou-se a instalação da engine do Docker utilizando o script oficial baixado por `curl -fsSL [https://get.docker.com](https://get.docker.com) -o get-docker.sh` e executado com `sudo sh get-docker.sh`. A correta instalação foi confirmada com `docker --version`. Para permitir o gerenciamento de containers sem a necessidade de privilégios de superusuário (sudo), o usuário pi foi adicionado ao grupo do Docker através do comando `sudo usermod -aG docker pi`. Após encerrar a sessão com `exit` e estabelecer uma nova conexão SSH (`ssh pi@rpi-104`), validou-se a permissão executando `docker run hello-world` sem o uso de sudo. Verificou-se a presença do plugin Docker Compose executando `docker compose version`. A organização dos serviços conteinizados foi estabelecida na estrutura de diretórios `~/fiscalizacao/portainer` e `~/fiscalizacao/node-red`. No diretório do Portainer [comando: `cd ~/fiscalizacao/portainer`], foi criado o arquivo `docker-compose.yml` utilizando o editor nano [comando: `nano docker-compose.yml`], definindo o serviço com a imagem `portainer/portainer-ce:latest`, escuta nas portas `8000:8000` e `9443:9443`, mapeamento do socket do Docker (`/var/run/docker.sock`), volume persistente `portainer_data` e o parâmetro `--admin-password` contendo o hash da senha padrão (147258369147). De forma análoga,

no diretório do Node-RED [comando: `cd ~/fiscalizacao/node-red`], criou-se seu respectivo `docker-compose.yml` [comando: `nano docker-compose.yml`] mapeando a imagem `nodered/node-red:latest`, a porta `1880:1880` e o volume `node_red_data`. Ambos os serviços foram iniciados via `docker compose up -d` em suas respectivas pastas e validados via `docker ps`. O acesso aos painéis web foi confirmado via navegador pelos endereços `https://rpi-104:9443` (Portainer, autenticado com o usuário admin) e `http://rpi-104:1880` (Node-RED). Para viabilizar a comunicação leve via MQTT entre os componentes da arquitetura, criou-se uma rede externa compartilhada no Docker por meio do comando `docker network create fiscalizacao`. Em seguida, estruturou-se o diretório do broker através de `mkdir -p ~/fiscalizacao/mqtt/config`. Criou-se o arquivo de configuração `config/mosquitto.conf` [comando: `nano config/mosquitto.conf`] com as diretivas `listener 1883`, `allow_anonymous true`, `persistence true`, `persistence_location /mosquitto/data/` e `log_dest file /mosquitto/log/mosquitto.log`. Na pasta `~/fiscalizacao/mqtt` [comando: `cd ~/fiscalizacao/mqtt`], elaborou-se o `docker-compose.yml` [comando: `nano docker-compose.yml`] utilizando a imagem `eclipse-mosquitto:latest`, exposta na porta `1883:1883`, com volumes para configuração, dados e logs, e vinculada à rede externa `fiscalizacao`. O serviço do broker foi subido via `docker compose up -d` e sua inicialização confirmada inspecionando os logs com `docker logs mosquitto --tail 20`, identificando a escuta na porta 1883 (Opening ipv4 listen socket on port 1883). Para permitir que o Node-RED se comunicasse com o Mosquitto através do nome do container, atualizou-se o `docker-compose.yml` do Node-RED em `~/fiscalizacao/node-red/` [comandos: `cd ~/fiscalizacao/node-red/` e `nano docker-compose.yml`], adicionando a seção de rede apontando para a rede externa `fiscalizacao`. A alteração foi aplicada reexecutando `docker compose up -d` no diretório do Node-RED. A validação de ponta a ponta foi realizada na interface web do Node-RED (`http://rpi-104:1880`). Configurou-se um fluxo de teste composto por um nó inject (configurado para enviar uma string como 42), conectado a um nó mqtt out configurado para o servidor `mosquitto:1883` no tópico padrão `fiscalizacao/<veiculo>/velocidade`. Paralelamente, um nó mqtt in foi apontado para o mesmo servidor e tópico, direcionando sua saída para um nó debug. Após realizar o Deploy e acionar o disparo no nó inject, a mensagem publicada trafegou pelo broker Mosquitto e foi exibida com sucesso na aba de depuração (debug), confirmando o funcionamento da cadeia inject -> mqtt out -> broker -> mqtt in -> debug. Por fim, os fluxos do Node-RED foram exportados via menu (Export) e todos os arquivos de configuração gerados (`docker-compose.yml` das aplicações e `mosquitto.conf`) foram versionados e commitados no repositório do grupo no GitHub [comandos: `git add .`, `git commit -m "Adiciona configurações e fluxos"` e `git push`].

Seguem-se os conteúdos dos arquivos:

1. `docker-compose.yml`

services:

node-red:

image: nodered/node-red:latest

container_name: node-red

restart: always

ports:

- "1880:1880"

volumes:

- node_red_data:/data

networks:

- fiscalizacao

volumes:

node_red_data:

networks:

fiscalizacao:

external: true

2. mosquitto.conf

services:

mosquitto:

image: eclipse-mosquitto:latest

container_name: mosquitto

restart: always

ports:

- "1883:1883"

volumes:

- ./config:/mosquitto/config

- mosquitto_data:/mosquitto/data

- mosquitto_log:/mosquitto/log

networks:

- fiscalizacao

volumes:

mosquitto_data:

mosquitto_log:

networks:

fiscalizacao:

external: true

Para realizar a conexão do ESP32 à Raspberry Pi, primeiro acessamos o terminal do Windows via Prompt de Comando e digitamos `ssh pi@rpi-104` para entrar na Raspberry, seguido do comando `ifconfig` na interface `wlan0` para consultar e identificar o endereço IP do broker MQTT (`10.10.10.104` naquele momento, porque ele muda todas as vezes que reinicia). Em seguida, preparamos a Arduino IDE conectado à internet, onde acessamos Arquivo > Preferências e colamos a URL [\[https://dl.espressif.com/dl/package_esp32_index.json\]](https://dl.espressif.com/dl/package_esp32_index.json)(https://dl.espressif.com/dl/package_esp32_index.json) no campo de placas adicionais. No Gerenciador de Placas, pesquisamos por `esp32` do Espressif Systems e realizamos a instalação; no Gerenciador de Bibliotecas, pesquisamos e instalamos a biblioteca `PubSubClient` de Nick O'Leary.

Ao conectar a placa ESP32 ao computador via cabo USB e selecionar a placa DOIT ESP32 DEVKIT V1, a Arduino IDE apresentou um erro de gravação (`exit status 2`) por mostrar apenas a porta `COM1`. Ao checar o Gerenciador de Dispositivos (`devmgmt.msc`), identificamos um aviso amarelo no item `CP2102 USB to UART Bridge Controller`, indicando a falta do driver de comunicação serial. A tentativa de atualização automática falhou devido ao bloqueio do Windows Update ("*Sua organização desativou atualizações automáticas*"), sendo necessário acessar o site da Silicon Labs, baixar o arquivo `.zip` do CP210x Universal Windows Driver, extrair os arquivos e instalar manualmente clicando com o botão direito no arquivo silabser.inf.

Com o driver instalado e a nova porta COM reconhecida (`COM3`), mudamos a conexão do computador para a rede Wi-Fi i4, inserimos o código final configurado para a rede `i4` (senha `147258369`) e para o IP da Raspberry, e clicamos em Carregar, mantendo o botão BOOT da placa pressionado durante a mensagem `Connecting.....` para iniciar a gravação. Por fim, abrimos o Monitor Serial ajustado para 115200 baud e pressionamos o botão EN/RST para confirmar a conexão do ESP32 ao Wi-Fi e ao broker MQTT, validando o envio intermitente

das velocidades (10, 90 e 150 km/h) publicadas no tópico [fiscalizacao/carro/velocidade](#) e o seu recebimento direto no painel do Node-RED.

Código usado no Arduino:

```
#include <WiFi.h>

#include <PubSubClient.h>

//
=====

// CONFIGURAÇÕES DA SUA REDE E SERVIDORES

//
=====

const char* ssid = "i4";           // Nome exato da sua rede Wi-Fi
const char* password = "147258369"; // Senha da rede Wi-Fi
const char* mqtt_server = "10.10.10.102"; // IP da Raspberry Pi (wlan0)
const int mqtt_port = 1883;        // Porta padrão do MQTT

//
=====

// VARIÁVEIS GLOBAIS E PINOS

//
=====

WiFiClient espClient;
PubSubClient client(espClient);

const int pinLED = 2;               // LED embutido do ESP32
int velocidades[] = {10, 90, 150}; // Simulação de velocidades
```

```

int indice = 0;                                // Controle do índice da lista

//
=====

// FUNÇÃO DE RECONEXÃO MQTT

//
=====

void reconectarMQTT() {

    while (!client.connected()) {

        Serial.print("Tentando conectar ao broker MQTT na Raspberry...");

        // Tenta conectar usando um ID único para a placa

        if (client.connect("ESP32_Fiscalizador")) {

            Serial.println("\n[SUCESSO] Conectado ao MQTT com sucesso!");

        } else {

            Serial.print("\n[ERRO] Falha na conexao. Codigo de erro: ");

            Serial.print(client.state());

            Serial.println(" | Tentando novamente em 3 segundos...");

            delay(3000);

        }

    }

}

//
=====

// CONFIGURAÇÃO INICIAL (SETUP)

```

```
//
=====

void setup() {

  pinMode(pinLED, OUTPUT);

  digitalWrite(pinLED, LOW);


  Serial.begin(115200);

  delay(1000);


  // 1. Conexão ao Wi-Fi

  Serial.println("\n-----");

  Serial.print("Iniciando conexao com Wi-Fi: ");

  Serial.println(ssid);

  WiFi.begin(ssid, password);


  // Imprime pontos enquanto tenta conectar no Wi-Fi

  while (WiFi.status() != WL_CONNECTED) {

    delay(500);

    Serial.print(".");

  }


  Serial.println("\n[SUCESSO] Wi-Fi Conectado!");

  Serial.print("Endereço IP do ESP32: ");

  Serial.println(WiFi.localIP());

  Serial.println("-----");
}
```

```

// 2. Configuração do Servidor MQTT

client.setServer(mqtt_server, mqtt_port);

}

//
=====

// LOOP PRINCIPAL

//
=====

void loop() {

    // Garante que o Wi-Fi continua conectado

    if (WiFi.status() != WL_CONNECTED) {

        Serial.println("Sinal do Wi-Fi perdido. Tentando reconectar...");

        WiFi.begin(ssid, password);

        while (WiFi.status() != WL_CONNECTED) {

            delay(500);

            Serial.print(".");

        }

    }

    // Garante que o MQTT continua conectado

    if (!client.connected()) {

        reconectarMQTT();

    }

    client.loop();

```

```
// Seleciona e formata o valor de velocidade atual

int velAtual = velocidades[indice];

char msg[10];

snprintf(msg, 10, "%d", velAtual);


// Envia a mensagem para o tópico configurado no Node-RED

Serial.print("Enviando velocidade para o Node-RED: ");

Serial.print(msg);

Serial.println(" km/h");


client.publish("fiscalizacao/carro/velocidade", msg);


// Executa o ciclo de piscar o LED (5 segundos aceso / 5 segundos apagado)

digitalWrite(pinLED, HIGH);

delay(5000);


digitalWrite(pinLED, LOW);

delay(5000);


// Avança para a próxima velocidade (10 -> 90 -> 150 -> 10...)

indice = (indice + 1) % 3;

}
```

Todos os arquivos usados como teste e como oficiais estão no GitHub. Abaixo seguem-se os prints e vídeos das etapas:

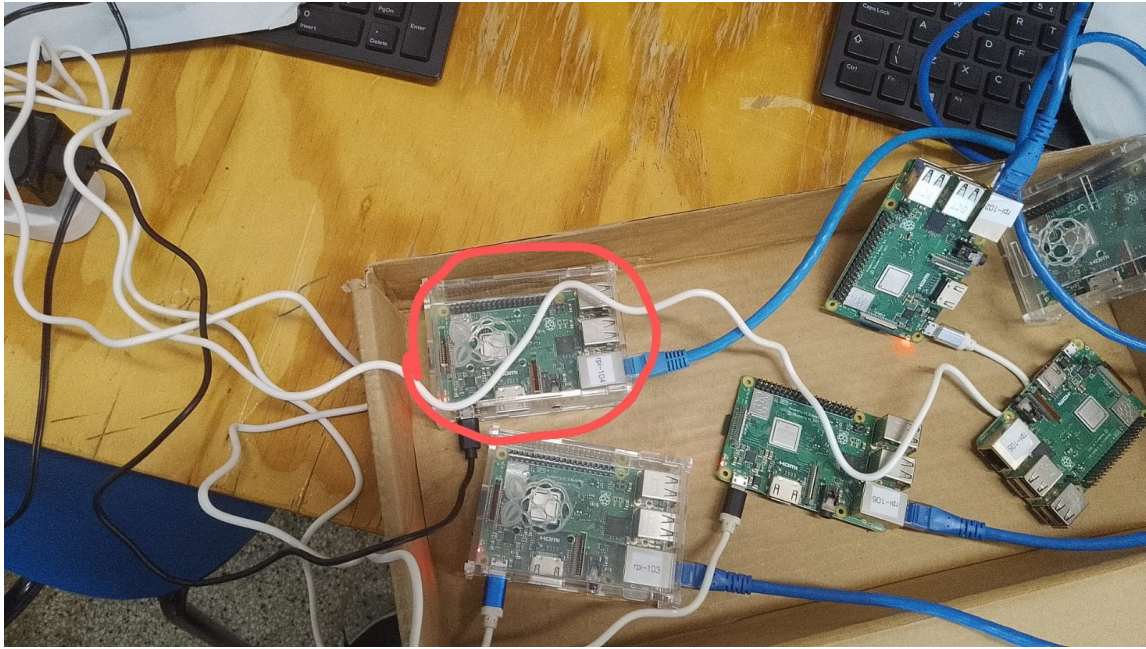
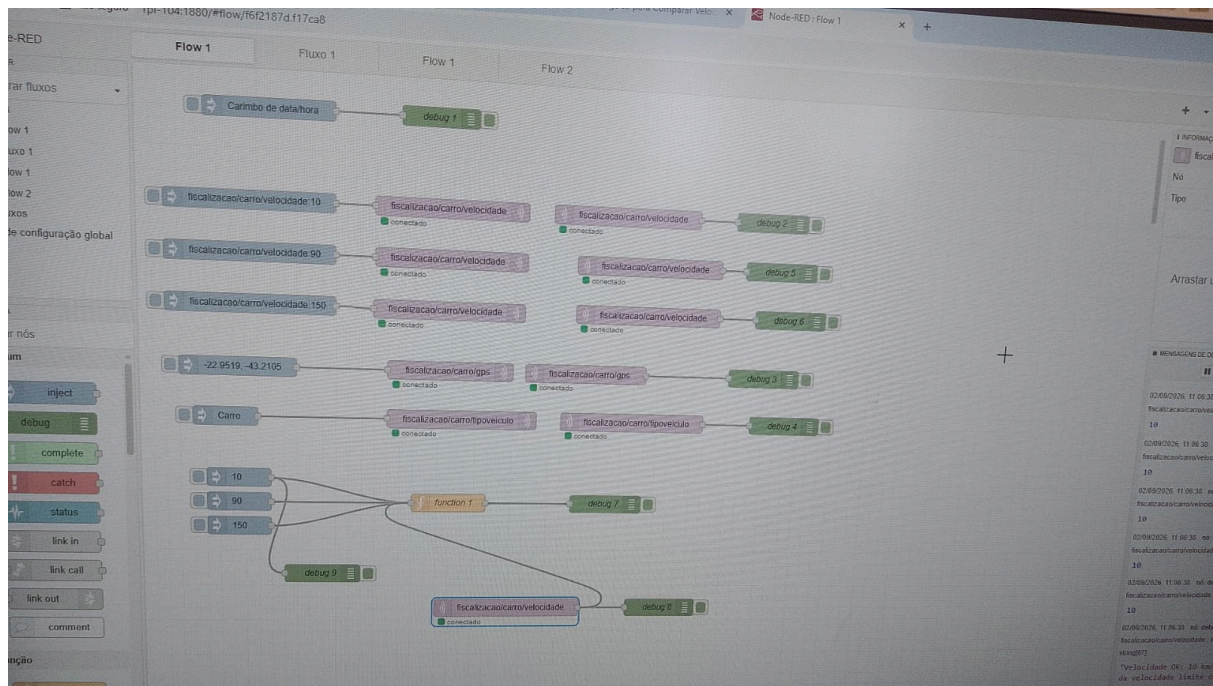


Imagem da rpi-104 (circulada em vermelho), a rpi que usamos na etapa final do projeto.



Roteador do Wi-Fi i4, Wi-Fi usado na comunicação entre a Rasp e a ESP.



Configuração final do ambiente NODE-Red.

<https://drive.google.com/drive/folders/1DJPoaQSx6dKKm3e5d0771t5uH3Udlhfz?usp=sharing>

Link do Drive com os vídeos da ESP piscando e o código das velocidades rodando no Arduino e no NODE-Red.

<https://canva.link/elhrb2k1a2ivi66>

Link do Slide de Apresentação.